Table Of Content

ZebraServo..... 2
SMotor2..... 4
SMotorPair..... 7
ZebraGyro..... 11
ZebraScreen..... 13
ZebraTOF..... 15
ZebraColour..... 17
ZebraHuskyLens..... 24

Library Details

ZebraServo

The **ZebraServo** library is designed to control standard servo motors on selected ESP32 GPIO pins. It provides a simple, port-based interface that allows users to move servos by specifying angles directly.

Internally, the library uses the **ESP32Servo** library for PWM signal generation, while handling port-to-GPIO mapping automatically. This makes servo control plug-and-play on supported Zebra ESP32 boards.

Supported Capabilities

- Controls standard servo motors using angle values
- Built on top of the ESP32Servo PWM library
- Maps predefined port numbers to valid ESP32 GPIO pins
- Simple and beginner-friendly API
- Designed for Zebra custom ESP32 boards

1. Include the Library

Add the following line at the top of your program:

```
#include <ZebraServo.h>
```

2. Create a Servo Object

Create a `ZebraServo` object using a valid port number.

```
ZebraServo servo(1);
```

Notes:

- A servo motor can be connected to 1 to 4 ports of motor and 1-3 ports of sensors.

- You may name the motor object anything you like.
- The motor object must be declared in the **global scope**, before setup() and loop().

3. Initialize the Servo

Initialize the servo inside the `setup()` function:

```
servo.begin();
```

Motor Control Functions

1.	**run_angles(int angle)** Runs the motor continuously at a given speed.	**Parameters:** `angle`: Desired servo angle (0–180 degrees) **Angle Reference** • `0` → Full right position • `90` → Center position • `180` → Full left position
	`servo.run_angles(180);`	

Example

```
#include <ZebraServo.h>

ZebraServo servo(1);

void setup() {
  servo.begin();
}

void loop() {
  servo.run_angles(180);
```

```
  delay(1000);

  servo.run_angles(135);
  delay(1000);

  servo.run_angles(90);
  delay(1000);

  servo.run_angles(45);
  delay(1000);

  servo.run_angles(0);
  delay(1000);

  servo.run_angles(90);
  delay(3000);
}
```

SMotor2

The **SMotor2** library is used to control a single DC motor connected to a motor port on the Zebra board. It provides essential motor control features, including speed control, directional movement, and precise motion using degrees, rotations, or time.

Supported Capabilities

- Run the motor forward or backward at a specified speed
- Stop the motor
- Rotate the motor by a specific number of degrees
- Rotate the motor by a specific number of rotations
- Run the motor for a fixed duration

1. Include the Library

Add the following line at the top of your program:

```
#include <SMotor2.h>
```

2. Create a Motor Object

Create a motor object by specifying the motor port number.

```
SMotor2 motor(1);
```

Notes:

- Replace 1 with 2, 3, or 4 based on the motor port used.
- This library works **only with motor ports** (not sensor ports).
- You may name the motor object anything you like.
- The motor object must be declared in the **global scope**, before setup() and loop().

3. Initialize the Motor

Initialize the motor inside the setup() function:

```
motor.begin();
```

Motor Control Functions

1.	**run_motor(int power)** Runs the motor continuously at a given speed.	**Parameters:** power: Motor speed (-100 to 100) - 0 stops the motor - Negative values rotate the motor backward - Positive values rotate the motor forward
	motor.run_motor(100); // Runs the motor forward at full speed	

2.	**stop_motor()**	Stop the motor immediately.
	motor.stop_motor();	

3.	**move_degrees(int degrees, int initial_speed)**
	
	Rotates the motor by a specific number of degrees.
		Parameters:
		
		• degrees: Number of degrees to rotate (e.g., 360)
		  ○ If degrees ≥ 360, the motor automatically decelerates near the end
		  ○ If degrees < 360, the motor runs at the minimum speed (20)
		
		• initial_speed: Starting speed (-100 to 100)
		  ○ Negative values rotate backward
		  ○ Positive values rotate forward

	motor.move_degrees(200, -80); // Rotates 200° backward at speed -80	

4.	**move_rotations(float rotations, int initial_speed)**
	
	Rotates the motor by a specific number of rotations.
		Parameters:
		
		• rotations: Number of rotations (e.g., 1.5)
		  ○ If rotations ≥ 1, the motor decelerates near completion
		  ○ If rotations < 1, the motor runs at the slowest speed
		• initial_speed: Starting speed (-100 to 100)
		  ○ Negative values rotate backward
		  ○ Positive values rotate forward

	motor.move_rotations(1.5, 65); // 1.5 rotations forward at speed 65	

5.	**move_time(float time, int initial_speed)**
	
	Runs the motor for a fixed duration.
		Parameters:
		
		• time: Duration in seconds (e.g., 2.5)
		• initial_speed: Motor speed (-100 to 100)
		  ○ Negative values rotate backward
		  ○ Positive values rotate forward

	motor.move_time(2.5, 100); // Runs forward for 2.5 seconds at speed 100	

SMotorPair

The **SMotorPair** library is used to control **two DC motors simultaneously**, most commonly for a robot's left and right drive wheels. It enables coordinated movement and steering, making it ideal for differential-drive robots.

Key Capabilities

- Drive forward or backward with steering control
- Turn left or right using a steering offset
- Move a fixed distance using encoder degrees or wheel rotations
- Run for a specific duration
- Stop both motors at the same time

1. Include the Library

Add the following line at the top of your program:

```
#include <SMotorPair.h>
```

2. Create a Motor Pair Object

Create a motor pair by specifying the motor ports used for the left and right motors.

```
SMotorPair robot(1, 2);
```

Notes:

- Replace 1 and 2 with the motor port numbers you are using.
- Only **motor ports** are supported (sensor ports will not work).
- The **first parameter** is the left motor, and the **second parameter** is the right motor.
- You may name the object anything you prefer.
- Declare the object in the **global scope**, before setup() and loop().

3. Initialize the Motor Pair

Initialize the motor pair inside the setup() function:

```
robot.begin();
```

Motor Control Functions

1.	**run_until(int steer, int initial_speed, bool (*stop_condition)())** Runs the motors continuously until a user-defined stop condition becomes true.

	Parameters:
	
	• steer: Steering offset (-100 to 100)
	  ○ 0 → straight
	  ○ Positive → curve right
	  ○ Negative → curve left
	• initial_speed: Speed (-100 to 100)

	○ 0 stops the motors ○ Negative values move backward ○ Positive values move forward • stop_condition: A boolean function that returns true when motion should stop *(Must be defined separately in the code)*
	robot.run_until(0, 50, stop_condition); // Move straight at speed 50 until condition is met
2.	**stop_motors()** Stop both motors immediately.
	robot.stop_motors();
3.	**move_degrees(int steer, int initial_speed, float degrees)** Moves both motors until a specified number of encoder degrees is reached.
	Parameters: • steer: Steering offset (-100 to 100) • initial_speed: Starting speed (-100 to 100) • degrees: Distance to move, measured in wheel encoder degrees **Behavior:** • Automatically decelerates when degrees ≥ 360 • Runs at minimum speed (20) when degrees < 360
	robot.move_degrees(45, -80, 720); // Curve right for 2 full wheel rotations at speed -80

4.	**move_rotations(int steer, int initial_speed, float rotations)** Moves both motors for a specified number of wheel rotations.
	Parameters: • steer: Steering offset (-100 to 100) • initial_speed: Starting speed (-100 to 100) • rotations: Number of wheel rotations **Behavior:** • Automatically decelerates when rotations ≥ 1 • Runs at minimum speed for values below 1 rotation
	`robot.move_rotations(0, 100, 1.5);  // Move straight for 1.5 wheel rotations`
5.	**move_time(int steer, int initial_speed, float time_seconds)** Runs both motors for a fixed duration.
	Parameters: • steer: Steering offset (-100 to 100) • initial_speed: Speed (-100 to 100) • time_seconds: Duration in seconds
	`robot.move_time(-30, 60, 1);  // Curve left for 1 second at speed 60`

ZebraGyro

The **ZebraGyro** library is designed for precise orientation tracking using the **MPU6050 gyroscope** in **DMP (Digital Motion Processor) mode**. It provides smooth and reliable yaw measurements and supports continuous angle tracking beyond 360°. The library is compatible with systems that use a **TCA9548A I²C multiplexer**, allowing multiple gyroscopes to be used on the same bus.

Key Capabilities

- Uses the MPU6050's DMP for accurate yaw calculation
- Tracks **continuous yaw rotation** (not limited to 0–360°)
- Supports multiple gyros via the TCA9548A I²C multiplexer
- Provides simple methods for updating, reading, and resetting yaw

1. Include the Library

Add the following line at the top of your program:

```
#include "ZebraGyro.h"
```

2. Create a Gyro Object

Create a ZebraGyro object using the predefined gyro port.

```
ZebraGyro gyro(7);
```

Notes:

- The value 7 is fixed and corresponds to the dedicated gyro port on the Zebra board.
- You may name the object anything you like.
- The object must be declared in the **global scope**, before setup() and loop().

Initialize the Gyro

Initialize the sensor inside the setup() function:

```
gyro.begin();
```

Gyroscope Functions

1.	**update()** Updates the internal yaw data by reading from the MPU6050's DMP.
	Usage Notes: • This function should be called **frequently inside loop()**. • Regular updates ensure accurate and responsive yaw tracking.
	`gyro.update();`
2.	**getYaw()** Returns the current **continuous yaw angle** in degrees.
	• The yaw value is not limited to 0–360°. • The angle will continue increasing or decreasing as the robot rotates.
	`float yaw = gyro.getYaw();`
3.	**resetYaw()**

	Resets the yaw reference to zero.
	Usage Notes: ● Call this function when you want to define a new zero-angle reference. ● Commonly used before starting a turn or navigation routine.
	gyro.resetYaw();

ZebraScreen

The ZebraScreen library is used to display text on an SSH1106 OLED screen connected through a TCA9548A I²C multiplexer. It is designed for simplicity and quick feedback, making it ideal for debugging, sensor readouts, and real-time system status messages.

Key Capabilities

- Supports SSH1106 OLED displays
- Works seamlessly with the TCA9548A I²C multiplexer
- Simple text-based output functions
- Line-based writing without clearing the full screen
- Useful for debugging and live data visualization

1. Include the Library

Add the following line at the top of your program:

```
#include "ZebraScreen.h"
```

2. Create a Screen Object

Create a ZebraScreen object using the predefined screen port.

```
ZebraScreen screen(0);
```

Notes:

- The value 0 is fixed and corresponds to the dedicated OLED screen channel on the Zebra board.
- You may name the object anything you like.
- The object must be declared in the **global scope**, before setup() and loop().

3. Initialize the Screen

Initialize the screen inside the setup() function:

```
screen.begin();
```

Screen Functions

1.	**write(String msg)** Displays a message starting from the **top-left corner** of the screen.
	Behavior: • Clears the display before writing • Always starts at the home position (x = 0, y = 0)
	screen.write("Hi Zebra!");
2.	**writeLine(uint8_t line, String msg)** Displays a message on a specific line without clearing the entire screen.

	Parameters: • line: Line number (0–7) ○ Each line is 8 pixels tall • msg: Text to display **Behavior:** • Text starts at x = 0 • Vertical position is based on the line number
	`screen.writeLine(3, "Voltage: 12.3V");`
3.	**clear()** Clears all text currently displayed on the screen.
	`screen.clear();`

ZebraTOF

The **ZebraTOF** library is designed for accurate distance measurement using **VL53L0X Time-of-Flight (TOF) sensors**. It works seamlessly with a **TCA9548A I²C multiplexer**, allowing multiple TOF sensors to be used on the same robot without address conflicts. The library provides simple methods for initializing sensors and reading real-time distance values in millimeters.

Key Capabilities

- Supports **VL53L0X Time-of-Flight distance sensors**
- Uses **TCA9548A I²C multiplexer** for multi-sensor setups
- Reads accurate distance values in **millimeters**

- Simple and beginner-friendly API
- Compatible with **Zebra Board sensor ports**

1. Include the Library

Add the following line at the top of your program:

```
#include "ZebraTOF.h"
```

2. Create a Screen Object

Create a ZebraTOF object using the sensor port number.

```
ZebraTOF leftTOF(1);
```

Notes:

- The value 1 represents the **sensor port number** (valid range: **1–6**).
- TOF sensors **must be connected to sensor ports** (motor ports are not supported).
- You can name the object anything you like (e.g., frontTOF, rightTOF).
- The object must be declared in the **global scope**, before setup() and loop().

3. Initialize the TOF Sensor

Initialize the sensor inside the setup() function:

```
leftTOF.begin();
```

ZebraTOF Functions

1.	**getDistance()**

	Returns the measured distance from the TOF sensor in **millimeters**.
	Behavior: • Distance values are updated in real time. • Best used inside the loop() function. • Ideal for obstacle detection, wall following, and navigation logic.
	int distance = leftTOF.getDistance();
2.	**Example Code**

```
#include "ZebraTOF.h"

ZebraTOF frontTOF(1);

void setup() {
  frontTOF.begin();
}

void loop() {
  int distance = frontTOF.getDistance();
  // Use distance for decision-making
}
```

ZebraColour

The **ZebraColour** library is designed to work with the **TCS34725 RGB color sensor** using a **TCA9548A I²C multiplexer**. It provides powerful and flexible color detection features for robotics applications such as **line following**, **color sorting**, and **environment sensing**.

The library supports both **full-color recognition** and **monochrome (black/white/gray) detection**, making it suitable for beginners as well as advanced robotics projects.

Key Capabilities

- Detects **11 colors** plus **black, white, gray, and no color**
- Supports **RGB and HSV color spaces**
- Dedicated **monochrome mode** for line following
- Automatic **I²C port switching** via TCA9548A
- Built-in **lux (brightness) measurement**
- Single-call data retrieval for high performance

1. Include the Library

Add the following line at the top of your program:

```
#include "ZebraColour.h"
```

2. Create a Sensor Object

Create a ZebraColour object using the sensor port number.

```
ZebraColour colorSensor(1);
```

Notes:

- Valid sensor ports: **0–7** (TCA9548A multiplexer)
- Must use a **sensor port** (not motor ports)
- You can name the object anything you like
- Declare the object in the **global scope**, before setup() and loop().

3. Initialize the Sensor

Initialize the sensor inside the setup() function:

```
void setup() {
  Serial.begin(115200);
  colorSensor.begin();
}
```

ZebraColour Functions

1.	**begin()** Initializes the color sensor with optimized default settings.
	colorSensor.begin();
2.	**readColour()** Reads raw color values and brightness from the sensor.
	Parameters: • r, g, b – RGB values (0–65535) • c – Clear channel value • lux – Calculated brightness **Behavior:** • Text starts at x = 0 • Vertical position is based on the line number
	void readColour(uint16_t &r, uint16_t &g, uint16_t &b, uint16_t &c, float &lux);

	Example

	`uint16_t r, g, b, c;`
	`float lux;`
	`colorSensor.readColour(r, g, b, c, lux);`

3.	**rgbToHSV()** Converts RGB values to HSV color space.
	`void rgbToHSV(uint16_t r, uint16_t g, uint16_t b,` `float &h, float &s, float &v);`
	Output: • h – Hue (0–360°) • s – Saturation (0.0–1.0) • v – Value/Brightness (0.0–1.0)
4.	**getColourID()** Detects color using HSV values and returns a color ID.
	`uint8_t getColourID(float h, float s, float v);`
	Supported Color IDs

ID	Color	Constant
0	No Color	COLOR_NONE
1	Black	COLOR_BLACK

2	White	COLOR_WHITE
3	Gray	COLOR_GRAY
4	Red	COLOR_RED
5	Orange	COLOR_ORANGE
6	Yellow	COLOR_YELLOW
7	Green	COLOR_GREEN
8	Cyan	COLOR_CYAN
9	Blue	COLOR_BLUE
10	Purple	COLOR_PURPLE
11	Magenta	COLOR_MAGENTA
255	Unknown	COLOR_UNKNOWN

5.

getColourName()

Converts a color ID into a human-readable name.

```
const char* getColourName(uint8_t colorID);
```

6.

getFullColourData() (Recommended)

	Retrieves all color data in one efficient call.
	`void getFullColourData(ColourData &data);`
	Provides • RGB + Clear values • HSV values • Lux (brightness) • Color ID
Monochrome (Line-Following) Mode	
7.	**detectMonochrome()** Detects **black, white, gray, or no color only**.
	`uint8_t detectMonochrome(float h, float s, float v);`
	Returns COLOR_UNKNOWN if a colored object is detected.
8.	**getGrayPercentage()** Returns how gray a surface is.
	`float getGrayPercentage(float v, float s);`
	• 0% → Black • 100% → White

	• -1 → Not grayscale
9.	**getMonochromeData()** Best for **line-following robots**
	`void getMonochromeData(MonochromeData &data);`
	Returns: • Color type (black/white/gray) • Gray percentage • Brightness and raw values
Data Structures	

```
struct ColourData {
  uint16_t r, g, b, c;
  float h, s, v;
  float lux;
  uint8_t colorID;
};

struct MonochromeData {
  uint16_t r, g, b, c;
  float h, s, v;
  float lux;
  uint8_t colorID;
  float grayPercentage;
};
```

Performance Tips

- Use getFullColourData() for fastest execution
- Minimum recommended delay: **50 ms**
- Optimal sensing distance: **5–10 mm**
- Typical current draw: **~0.2 mA**

ZebraHuskyLens

The **ZebraHuskyLens** library provides an easy-to-use interface for the **HuskyLens Vision Sensor**. This documentation focuses on **Color Recognition** and **Object Detection & Tracking** modes, allowing robots to detect, identify, and track multiple objects using learned color tags.

Key Features

- Object detection and tracking of multiple objects simultaneously
- Color recognition using learned color tags (ID 1–7)
- Access to object position data (X, Y coordinates)
- Support for multiple detected objects via indexing

Wiring

HuskyLens Pin	ESP32 Pin	Description
T (TX)	GPIO 4 (RX)	Data from HuskyLens to ESP32
R (RX)	GPIO 0 (TX)	Data from ESP32 to HuskyLens
VCC	5V	Power

GND	GND	Ground

Pin Configuration Options

- **Option 1 (Default):** RX = 4, TX = 0
- **Option 2 (Alternative):** RX = 19, TX = 5

Note: Any available GPIO pins may be used for RX/TX. The selected pins must match the constructor parameters.

1. Include the Library

```
#include "ZebraHuskyLens.h"
```

2. Create a HuskyLens Object

Option 1 (Default pins):

```
ZebraHuskyLens husky(4, 0);
```

Option 2 (Alternative pins):

```
ZebraHuskyLens husky(19, 5);
```

Parameters

- First parameter: RX pin (ESP32 receives data from HuskyLens)
- Second parameter: TX pin (ESP32 sends data to HuskyLens)

The object must be declared in the global scope, before setup() and loop().

3. Initialize the Sensor

Initialize the sensor inside the setup() function:

```
void setup() {
  Serial.begin(115200);
```

```
  if (!husky.begin()) {
    Serial.println("Failed to initialize HuskyLens!");
    while (1);
  }

  Serial.println("HuskyLens Ready!");
}
```

4. Update and Read Data

```
void loop() {
  if (husky.update()) {

    if (husky.isDetected()) {
      int x = husky.getX();
      int y = husky.getY();
      Serial.printf("Object at X=%d, Y=%d\n", x, y);
    }
  }

  delay(100);
}
```

Notes

- Call update() inside loop() before reading any data.
- Use isLearned() to confirm the HuskyLens has learned at least one object (when required by the selected algorithm).

ZebraTOF Functions

1.

Initialization and Update

begin()

Initializes communication with the HuskyLens.

Returns

- true if connected successfully
- false if connection failed

```
if (!husky.begin()) {
  Serial.println("HuskyLens not found!");
  while (1);
}
```

2.

update()

Requests and reads the latest data from HuskyLens. Must be called before reading any object results

Returns

- true if connected successfully
- false if connection failed

```
if (husky.update()) {
  if (husky.isDetected()) {
    Serial.println("Object detected!");
  }
}
```

Status Check Functions	
3.	**isLearned()** Checks whether the HuskyLens has learned at least one object/color (depends on selected algorithm mode). **Returns** • true if at least one object is learned • false if nothing is learned
	Example

	`if (!husky.isLearned()) {`
	`  Serial.println("Please train HuskyLens first!");`
	`  return;`
	`}`

4.	**isDetected()** Checks whether any object is currently detected. **Returns** • true if one or more objects detected • False if nothing detected
	Example
	`if (husky.isDetected()) {`

```
  Serial.println("Object found!");
} else {
  Serial.println("No objects detected");
}
```

5.

getObjectCount()

Returns the number of objects currently detected.

Returns

- Total detected objects (0 or more)

Example

```
int count = husky.getObjectCount();
Serial.printf("Detected %d objects\n", count);
```

Basic Object Data (First Object)

getX()

Returns X coordinate of the first detected object.

Returns

- X position in pixels (0–319)
- Screen center is approximately X = 160

getY()

Returns Y coordinate of the first detected object.

Returns

- Y position in pixels (0–239)
- Screen center is approximately Y = 120

getID()

Returns the learned ID of the first detected object.

Returns

- ID number (example: 1–7 for color tags, or learned object/face IDs depending on mode)

getWidth()

Returns width of the first detected object (pixels).

getHeight()

Returns height of the first detected object (pixels).

Multi-Object and Advanced Functions

When multiple objects are detected, use indexed access.

getX(int index)

Returns the X coordinate of the object at the specified index.

getY(int index)

Returns the Y coordinate of the object at the specified index.

getID(int index)

Returns the ID of the object at the specified index.

Index Range:

0 to getObjectCount() - 1

Example 1: Color Recognition

```
#include "ZebraHuskyLens.h"

ZebraHuskyLens husky(4, 0);

void setup() {
  Serial.begin(115200);

  if (!husky.begin()) {
    Serial.println("HuskyLens initialization failed!");
    while (1);
  }

  Serial.println("Color Recognition Ready");
}

void loop() {
  if (husky.update()) {
    if (husky.isDetected()) {
      int id = husky.getID();
      Serial.print("Color Tag ID: ");
      Serial.println(id);
      Serial.printf("Position: X=%d, Y=%d\n", husky.getX(), husky.getY());
    }
  }
  delay(100);
}
```

Example 2: Multiple Object Detection

```
#include "ZebraHuskyLens.h"

ZebraHuskyLens husky(4, 0);

void setup() {
  Serial.begin(115200);

  if (!husky.begin()) {
    Serial.println("HuskyLens initialization failed!");
    while (1);
  }

  Serial.println("Multiple Object Detection Ready");
}

void loop() {
  if (husky.update()) {
    int count = husky.getObjectCount();

    if (count > 0) {
      Serial.printf("Detected %d objects\n", count);

      for (int i = 0; i < count; i++) {
        Serial.printf(
          "Object %d: ID=%d, X=%d, Y=%d\n",
          i + 1,
          husky.getID(i),
          husky.getX(i),
          husky.getY(i)
        );
      }
    }
  }
  delay(500);
}
```

Training HuskyLens

Color Recognition Mode

1. Power on HuskyLens
2. Select **Color Recognition** mode
3. Point the camera at the colored object
4. Press the button once (bounding box appears)
5. Press the button again to confirm and assign a tag (ID 1–7)

Object Detection Mode

1. Power on HuskyLens
2. Select Object Tracking mode
3. Point the camera at the object
4. Press the button once (bounding box appears)
5. Press the button again to confirm